

Pontifícia Universidade Católica de Campinas

ESTUDOS DE BANCO DE DADOS I

Prof Dra Renata Arantes

E-mail: renata.arantes@puc-campinas.edu.br

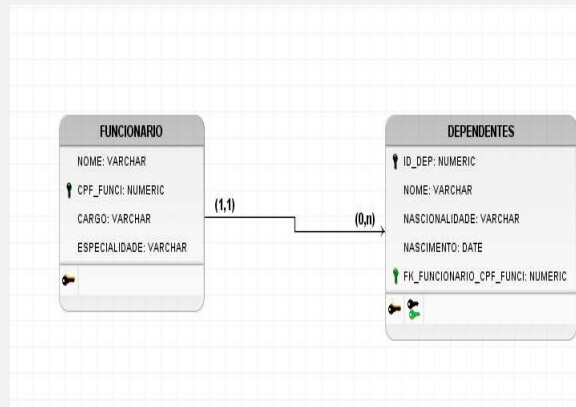
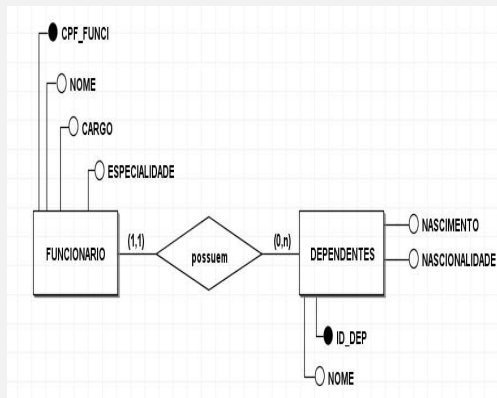
Por que mapear?

- Transformar a lógica do negócio em estrutura física
- Preparar o banco de dados para implementação
- Garantir consistência e clareza

Mapeamento do esquema Conceitual para o Relacional

- **Objetivo:** Reforçar as diferenças entre os dois níveis e o papel de cada um no design de bancos de dados.
- **Conceitos-chave:**
 - Esquema conceitual: uma visão abstrata, normalmente representada por diagramas ER (Entidade-Relacionamento).
 - Esquema relacional: a implementação do modelo conceitual usando tabelas, chaves e relacionamentos.
- **Exemplo visual:** Apresentar um diagrama MER simples e a correspondente representação em um modelo relacional (tabelas).

Conceitual / Lógico / Relacional



```
Página de Boas-Vindas | Renata

Planilha | Query Builder

create table funcionario
( cpf_func number (13) CONSTRAINT pk_cpf_func primary key,
  nome varchar2(30),
  cargo varchar2(30),
  especialidade varchar2(30));

create table dependentes
( id_dep number (4) CONSTRAINT pk_id_dep primary key,
  nome varchar2(30),
  nascimento date,
  nacionalidade varchar2(30),
  cpf_fun_dep number(13),
  foreign key (cpf_fun_dep) references funcionario (cpf_func));
```

Saída do Script x

Tarefa concluída em 0,043 segundos

*Action:

Table FUNCIONARIO criado.

Table DEPENDENTES criado.

Etapas do Mapeamento

a) Entidades Simples -> Tabelas: - Cada entidade se torna uma tabela.

- Atributos simples viram colunas.
- Definir uma chave primária (Primary Key).

b) Atributos compostos -> Colunas Separadas:

- Atributos compostos devem ser desmembrados em múltiplas colunas.

c) Relacionamentos:

- **1:1:** Pode ser implementado como uma Foreign Key em uma das tabelas.
- **1:N :** O lado "Muitos" recebe uma Foreign Key da tabela do lado
- **N:N :** Criação de uma tabela associativa contendo as Foreign Keys de ambas as tabelas envolvidas no relacionamento.

- **d) Entidades fracas -> Tabelas com dependência:** - Adicionar uma Foreign Key que aponta para a tabela da entidade forte associada.

Etapas do Mapeamento

- **Casos Especiais**
- **Atributos multivalorados:** Criar uma tabela separada para armazenar múltiplos valores, com uma Foreign Key para a tabela original.
- **Generalização/especialização:** As opções (uma tabela para cada especialização ou tabela única com discriminação).

Estratégias:

- 1. Uma tabela única com atributo discriminador
- 2. Superclasse + Subclasses
- 3. Apenas tabelas das subclasses

Entidade Associativas

- Uma entidade associativa (ou tabela associativa) precisa de uma chave primária (Primary Key), assim como qualquer tabela em um banco de dados relacional.
- **Existem duas abordagens comuns para definir a chave primária em uma entidade associativa:**

1. Chave Primária Composta

- A chave primária da tabela associativa é formada pela combinação das chaves estrangeiras (Foreign Keys) das tabelas que ela está associando.

2. Chave Primária Substituta

- Além das Foreign Keys, a entidade associativa pode ter uma chave primária substituta, ou seja, um atributo adicional (como um id) criado especificamente para ser a chave primária. Nesse caso, as chaves estrangeiras ainda são restritas por UNIQUE para evitar duplicações.

•**Exemplo Chave Composta:** Considere as entidades **Estudante** e **Curso** com um relacionamento N (muitos para muitos).

A entidade associativa **Inscrição** terá uma chave primária composta pelas chaves estrangeiras (id_estudante e id_curso).

```
CREATE TABLE Inscrição (  
    id_estudante INT,  
    id_curso INT,  
    data_inscricao DATE,  
    PRIMARY KEY (id_estudante, id_curso),  
    FOREIGN KEY (id_estudante) REFERENCES Estudante(id_estudante),  
    FOREIGN KEY (id_curso) REFERENCES Curso(id_curso)  
);
```

- **Exemplo Chave Composta:** (MELHORADA)

Pode colocar NOT NULL nos campos da chave composta, porque PK já exige que não sejam nulos.

Se quiser, pode adicionar ON DELETE CASCADE nas FKs, para que, se um curso ou estudante for apagado, a inscrição associada também seja removida.

Exemplo com essas melhorias:

```
CREATE TABLE Inscricao (  
    id_estudante INT NOT NULL,  
    id_curso INT NOT NULL,  
    data_inscricao DATE,  
    PRIMARY KEY (id_estudante, id_curso),  
    FOREIGN KEY (id_estudante) REFERENCES Estudante(id_estudante) ON DELETE CASCADE,  
    FOREIGN KEY (id_curso) REFERENCES Curso(id_curso) ON DELETE CASCADE  
);
```

Criando tabelas principais

```
CREATE TABLE Estudante (  
  id_estudante INT PRIMARY KEY,  
  nome VARCHAR2(100)  
);
```

```
CREATE TABLE Curso (  
  id_curso INT PRIMARY KEY,  
  nome VARCHAR2(100)  
);
```

Criando tabela associativa com chave composta

```
CREATE TABLE Inscricao (  
  id_estudante INT NOT NULL,  
  id_curso INT NOT NULL,  
  data_inscricao DATE,  
  PRIMARY KEY (id_estudante, id_curso),  
  FOREIGN KEY (id_estudante) REFERENCES Estudante(id_estudante),  
  FOREIGN KEY (id_curso) REFERENCES Curso(id_curso)  
);
```

Inserindo dados nas tabelas principais

```
-- Estudantes  
INSERT INTO Estudante VALUES (1, 'Ana');  
INSERT INTO Estudante VALUES (2, 'Bruno');  
INSERT INTO Estudante VALUES (3, 'Carla');  
  
-- Cursos  
INSERT INTO Curso VALUES (10, 'Banco de Dados');  
INSERT INTO Curso VALUES (20, 'Programação');  
INSERT INTO Curso VALUES (30, 'Engenharia de Software');
```

Inserindo dados na tabela associativa (Inscrição)

```
-- Ana se inscreveu em Banco de Dados  
INSERT INTO Inscricao VALUES (1, 10, DATE '2025-09-01');  
  
-- Ana também se inscreveu em Programação  
INSERT INTO Inscricao VALUES (1, 20, DATE '2025-09-02');  
  
-- Bruno se inscreveu em Banco de Dados  
INSERT INTO Inscricao VALUES (2, 10, DATE '2025-09-01');  
  
-- Carla se inscreveu em Engenharia de Software  
INSERT INTO Inscricao VALUES (3, 30, DATE '2025-09-03');
```

```
SELECT e.nome    AS estudante,  
       c.nome    AS curso,  
       i.data_inscricao  
FROM   Inscricao i  
JOIN   Estudante e ON i.id_estudante = e.id_estudante  
JOIN   Curso c    ON i.id_curso = c.id_curso  
ORDER BY e.nome, c.nome;
```

Exemplo de saída (com base nos INSERTs que montamos):

ESTUDANTE	CURSO	DATA_INSCRICAO
-----------	-------	----------------

Ana	Banco de Dados	01-SEP-2025
-----	----------------	-------------

Ana	Programação	02-SEP-2025
-----	-------------	-------------

Bruno	Banco de Dados	01-SEP-2025
-------	----------------	-------------

Carla	Engenharia de Software	03-SEP-2025
-------	------------------------	-------------

•Exemplo Chave Substituta:

```
CREATE TABLE Inscrição_always (  
    id_inscricao NUMBER GENERATED ALWAYS AS IDENTITY  
    id_estudante NUMBER,  
    id_curso NUMBER,  
    data_inscricao DATE,  
    PRIMARY KEY (id_inscricao),  
    UNIQUE (id_estudante, id_curso),  
    FOREIGN KEY (id_estudante) REFERENCES  
Estudante(id_estudante),  
    FOREIGN KEY (id_curso) REFERENCES Curso(id_curso)  
);
```

Observação: O Oracle sempre gera o valor

```
CREATE TABLE Inscricao_default (  
    id_inscricao NUMBER GENERATED BY DEFAULT AS  
IDENTITY (START WITH 100 INCREMENT BY 10),  
    id_estudante NUMBER,  
    id_curso NUMBER,  
    data_inscricao DATE,  
    PRIMARY KEY (id_inscricao),  
    UNIQUE (id_estudante, id_curso),  
    FOREIGN KEY (id_estudante) REFERENCES  
Estudante(id_estudante),  
    FOREIGN KEY (id_curso) REFERENCES  
Curso(id_curso)  
);
```

Observação: O oracle gera sozinho, mas pode inserir valor manualmente

```
INSERT INTO Inscricao_Always (id_estudante, id_curso, data_inscricao)  
VALUES (1, 10, DATE '2025-09-22');
```

```
INSERT INTO Inscricao_Default (id_estudante, id_curso, data_inscricao)  
VALUES (1, 10, DATE '2025-09-22');
```

O que é uma ENTIDADE FRACA

- Uma **entidade fraca** é uma entidade que não pode ser identificada unicamente por seus próprios atributos e depende de uma **entidade forte** (ou proprietária) para ser identificada. Diferente de uma entidade forte, que tem uma chave primária própria, a entidade fraca depende de uma **chave estrangeira** (Foreign Key) da entidade forte e de um **identificador parcial** para formar sua chave primária.

Características de uma Entidade Fraca:

- **Dependência de existência:** A entidade fraca não pode existir sem a entidade forte. Se a entidade forte for removida, a entidade fraca também deve ser removida.
- **Chave Primária Composta:** A chave primária da entidade fraca é composta pela chave estrangeira da entidade forte mais um identificador parcial próprio da entidade fraca.
- **Relacionamento identificador:** A entidade fraca tem um relacionamento especial com a entidade forte, chamado **relacionamento identificador**, que indica que a entidade fraca depende da entidade forte.

Diagrama ER de uma ENTIDADE FRACA

- Na notação ER tradicional, uma **entidade fraca** é representada por um **retângulo duplo** e o **relacionamento identificador** por um **losango duplo**, enquanto uma entidade forte é representada por um retângulo simples.
- **Departamento** (forte) → representado por um retângulo simples.
- **Projeto** (fraca) → representado por um retângulo duplo.
- **Relacionamento identificador** entre eles → losango duplo.



Entidade fraca – como fazer

- **Exemplo de ER com Entidade Fraca**

- 1. Departamento e Projeto**

- **Departamento (entidade forte):** id_departamento, nome_departamento
- **Projeto (entidade fraca):** Um projeto não pode ser identificado sem saber a qual departamento ele pertence. A chave primária de Projeto será uma combinação de id_departamento (chave estrangeira) e id_projeto (identificador parcial).
- **Entidade Forte:** A entidade forte possui sua chave primária definida normalmente.
- **Exemplo:** Se a entidade forte for **Departamento** com id_departamento como chave primária.

Entidade Fraca:

- A entidade fraca, que depende da entidade forte, terá uma chave composta pela chave primária da entidade forte (como chave estrangeira) mais o identificador parcial da entidade fraca.

Entidade fraca – como fazer

- Exemplo: Se a entidade fraca for **Projeto** dependente de **Departamento**, a chave primária de Projeto será composta por **id_departamento** (chave estrangeira) e **id_projeto** (identificador parcial da entidade fraca).

```
CREATE TABLE Projeto (  
    id_departamento INT,  
    id_projeto INT,  
    nome_projeto VARCHAR(255),  
    PRIMARY KEY (id_departamento, id_projeto),  
    FOREIGN KEY (id_departamento) REFERENCES Departamento (id_departamento)  
);
```

Observações:

- A chave estrangeira (id_departamento) é usada para garantir a ligação com a entidade forte.
 - O id_projeto é o identificador parcial que, junto com id_departamento, forma a chave primária.
- Dessa forma, a entidade fraca é sempre dependente da entidade forte para sua identificação.

Generalização/Especialização

- A **generalização/especialização** é um conceito em modelagem de dados que reflete uma hierarquia entre entidades, onde uma entidade mais genérica (superclasse) é subdividida em entidades mais específicas (subclasses). Esse tipo de modelagem é útil quando algumas entidades compartilham características em comum, mas também possuem características próprias que as distinguem.
- **Definições:**
- **Generalização:** Processo de abstração onde características comuns de várias entidades são extraídas para criar uma entidade mais genérica.
- **Especialização:** Processo inverso à generalização, onde uma entidade genérica é subdividida em entidades mais específicas, com características adicionais.

Generalização/Especialização

Exemplo : CLIENTE (Generalização)

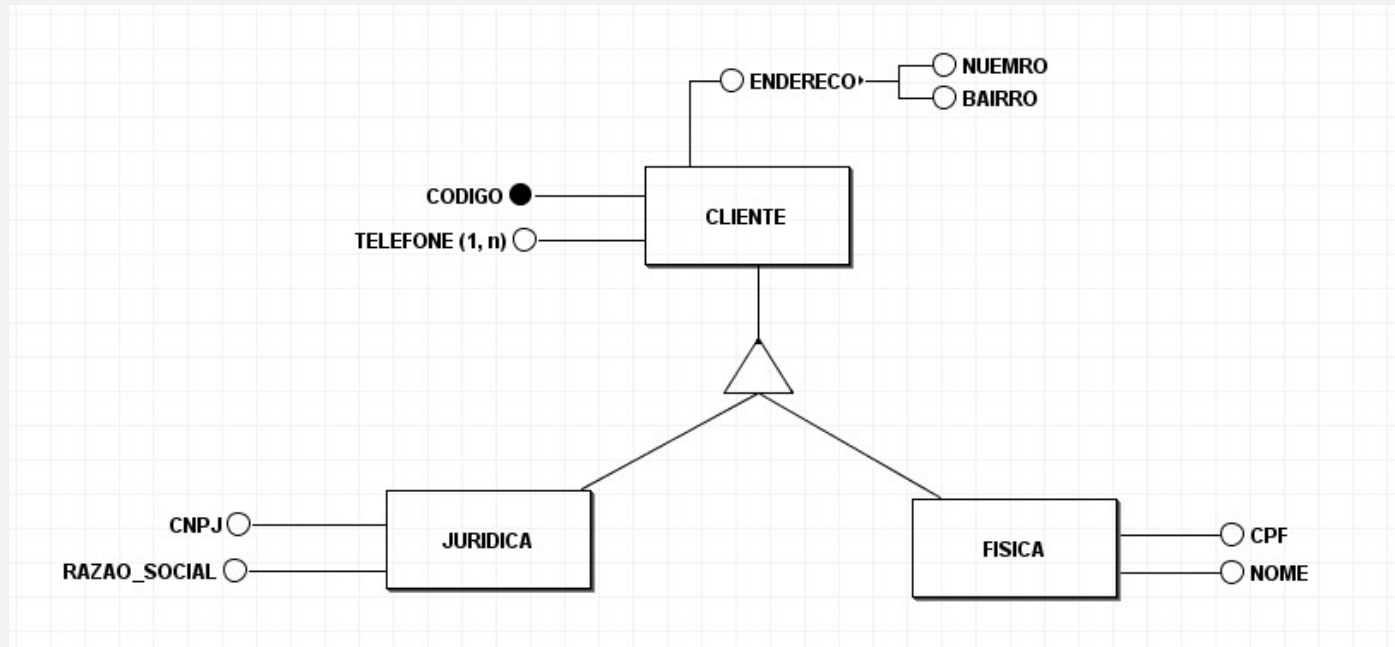
Imagine que temos uma entidade CLIENTE e queremos especializá-la em dois tipos: JURIDICA E FISICA

Entidade Genérica:

- CLIENTE**: código, telefone, endereço

Entidades Especializadas:

- Jurídica**: cnpj, razão social
- Física**: cpf, atributo



Generalização/Especialização

Outro Exemplo : Funcionário (Generalização)

Imagine que temos uma entidade Funcionário e queremos especializá-la em dois tipos: Gerente e Operário.

Entidade Genérica:

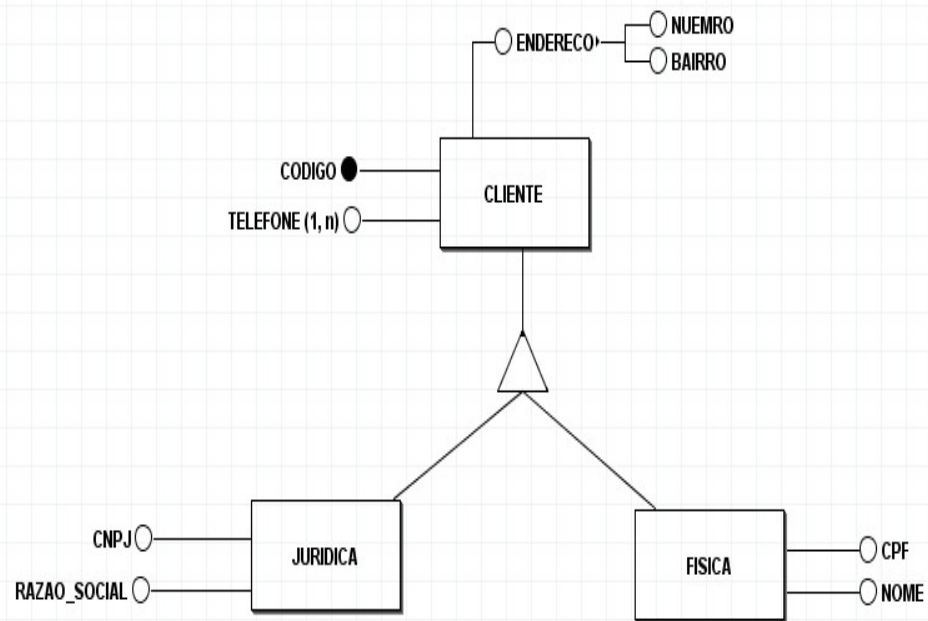
- **Funcionário:** id_funcionario, nome, salario

Entidades Especializadas:

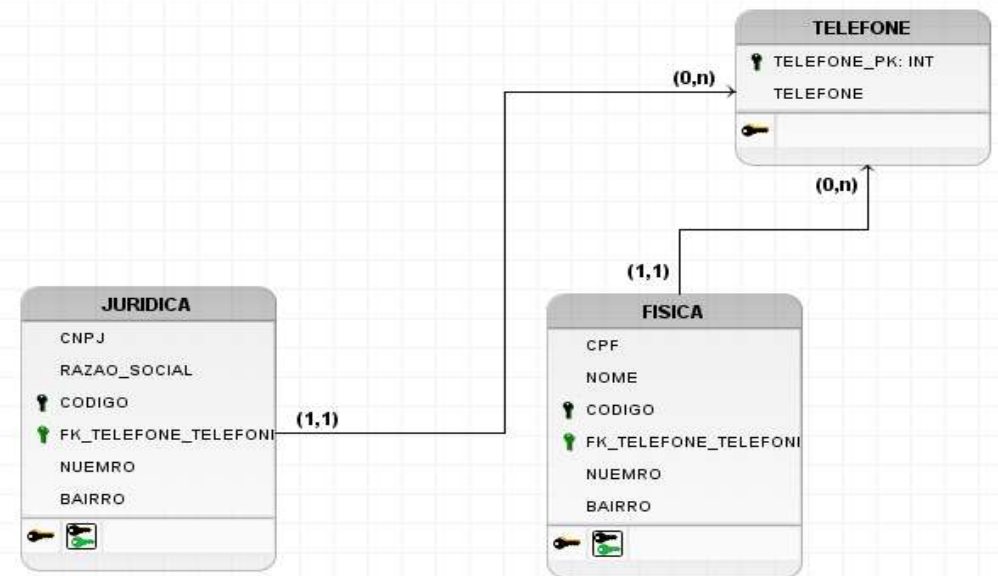
- **Gerente:** id_funcionario, bonus
- **Operário:** id_funcionario, horas_trabalhadas

O uso da Generalização é indicado quando existe Atributos comuns a varias entidades , neste caso criamos uma Entidade Pai / Super Classe que contenha os atributos comuns às outras entidades especializadas

CONCEITUAL



LOGICO



Implementação no Modelo Relacional

Existem três formas comuns de implementar a generalização/especialização no modelo relacional:

1. Tabela única para a entidade genérica e todas as subclasses:

- Usar uma única tabela (Funcionário) que inclui todos os atributos, incluindo os específicos das subclasses. Um campo adicional pode ser usado para discriminar o tipo (ex: "Gerente" ou "Operário").
- **Desvantagem:** Pode haver muitos campos nulos, especialmente se houver muitas especializações.

```
CREATE TABLE Funcionario (  
    id_funcionario INT PRIMARY KEY,  
    nome VARCHAR(255),  
    salario DECIMAL,  
    bonus DECIMAL,    -- Para gerente  
    horas_trabalhadas INT -- Para operário  
);
```

Implementação no Modelo Relacional

Existem três formas comuns de implementar a generalização/especialização no modelo relacional:

2. Tabelas separadas para cada entidade especializada:

- Criar uma tabela para cada entidade especializada, incluindo os atributos genéricos da entidade pai.
- **Desvantagem:** Pode haver duplicação de dados.

```
CREATE TABLE Gerente (  
    id_funcionario INT PRIMARY KEY,  
    nome VARCHAR(255),  
    salario DECIMAL,  
    bonus DECIMAL  
);
```

```
CREATE TABLE Operario (  
    id_funcionario INT PRIMARY KEY,  
    nome VARCHAR(255),  
    salario DECIMAL,  
    horas_trabalhadas INT  
);
```

Implementação no Modelo Relacional

Existem três formas comuns de implementar a generalização/especialização no modelo relacional:

- **3. Tabelas separadas para a entidade genérica e cada subclasse:**
- Criar uma tabela para a entidade genérica (com atributos comuns) e tabelas separadas para cada entidade especializada (apenas com os atributos específicos).
- **Vantagem:** Reduz a duplicação e o desperdício de espaço, mantendo a estrutura lógica.

```
CREATE TABLE Funcionario (  
  id_funcionario INT PRIMARY KEY,  
  nome VARCHAR(255),  
  salario DECIMAL  
);
```

```
CREATE TABLE Gerente (  
  id_funcionario INT PRIMARY KEY,  
  bonus DECIMAL,  
  FOREIGN KEY (id_funcionario) REFERENCES Funcionario(id_funcionario)  
);
```

```
CREATE TABLE Operario (  
  id_funcionario INT PRIMARY KEY,  
  horas_trabalhadas INT,  
  FOREIGN KEY (id_funcionario) REFERENCES Funcionario(id_funcionario)  
);
```